# SIMATS ENGINEERING
## ASSESSMENT TOOL 1

**COURSE CODE: ITA 06** **COURSE NAME: MACHINE LEARNING**

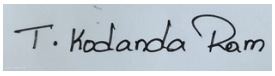
**COURSE OUTCOME COVERED**
***CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)*** *Assessment Tool Weightage: 40%*

**QUESTIONS: 15** **Total Marks:25**

Annexure A

## Experiments

***Code of Conduct:***
*I T . KODANDA RAM (192524162) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:*** 

Annexure A

**Short Answer Test**

1, Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

2. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.

3. Implement the Expectation-Maximization (EM) algorithm for a mixture model and conduct experiments to study its convergence behavior. Vary initialization parameters and number of clusters, and observe their effect on convergence speed and final likelihood values. Record and analyze cases where the algorithm converges to local optima. Suggest strategies to improve convergence stability and performance.

4. Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.

5. Implement a concept learning algorithm (such as Candidate Elimination) and perform experiments to analyze the effect of hypothesis space size on learning performance. Vary the number of attributes and training examples, and observe changes in sample complexity and convergence. Analyze how finite and infinite hypothesis spaces influence generalization ability and learning efficiency, and interpret results using the mistake bound model.

6. Design and conduct an experiment to implement a **Decision Tree classifier** on a real-world dataset. Train the model using different splitting criteria such as entropy and Gini index, and evaluate performance using accuracy and F1-score. Analyze the

impact of tree depth and pruning on overfitting, and compare results across different configurations.

7. Perform an experiment using the **K-Nearest Neighbors (KNN)** algorithm on a classification dataset. Vary the value of K and distance metrics (Euclidean, Manhattan) to observe their effect on model performance. Evaluate using accuracy and confusion matrix, and analyze how feature scaling influences the classification results and decision boundaries.

8. Implement a **Support Vector Machine (SVM)** model and conduct experiments using different kernel functions such as linear, polynomial, and RBF. Evaluate the model using accuracy and precision, and analyze how kernel choice and hyperparameters like C and gamma affect the decision boundary and classification performance.

9. Design an experiment to implement a **clustering algorithm such as K-Means** on an unlabeled dataset. Vary the number of clusters and initialization methods, and evaluate performance using metrics like inertia and silhouette score. Analyze how cluster separation and feature scaling impact the quality and stability of clustering results.

10. Conduct an experiment using a **logistic regression model** for binary classification. Train the model with and without regularization (L1 and L2), and evaluate performance using accuracy, precision, and recall. Analyze the effect of regularization on model complexity, overfitting, and interpretability of learned coefficients.

11. Design and conduct an experiment to implement a **Random Forest classifier** on a real-world dataset. Train the model with varying numbers of trees and maximum depth. Evaluate performance using accuracy, precision, recall, and F1-score. Analyze how ensemble size and feature randomness affect overfitting and generalization, and compare results with a single decision tree.

12. Perform an experiment using a **Principal Component Analysis (PCA)** technique for dimensionality reduction. Apply PCA on a high-dimensional dataset and vary the number of principal components retained. Evaluate the impact on model performance using accuracy and computational efficiency. Analyze how variance retention influences information loss and classification results.

13. Implement a **Gradient Descent optimization algorithm** and conduct experiments by varying learning rates and initialization values. Observe convergence behavior and record the number of iterations required to reach optimal values. Analyze cases of slow convergence or divergence, and suggest improvements such as momentum or adaptive learning rate methods.

14. Design an experiment to implement a **Neural Network classifier** for a classification task. Vary the number of hidden layers, neurons, and activation functions. Evaluate the model using accuracy and loss curves, and analyze how network architecture affects performance, convergence, and overfitting.

15. Conduct an experiment using a **hierarchical clustering algorithm** on an unlabeled dataset. Compare different linkage methods such as single, complete, and average linkage. Evaluate clustering quality using dendrogram analysis and silhouette score. Analyze how linkage criteria influence cluster formation and interpretability of results.

## RUBRICS FOR CALCULATION

| Criteria | Weightage | Level 4 – Exemplary (5 Marks) | Level 3 – Proficient (4 Marks) | Level 2 – Developing (2–3 Marks) | Level 1 – Beginning (0–1 Mark) |
|---|---|---|---|---|---|
| **Conceptual Understanding** | **30** | Demonstrates clear and complete understanding of the concept | Good understanding with minor gaps | Basic understanding with some misconceptions | Poor or incorrect understanding |
| **Accuracy of Answer** | **25** | Completely correct answer with no errors | Mostly correct with minor errors | Partially correct with noticeable errors | Incorrect or irrelevant answer |
| **Application of Knowledge** | **15** | Correctly applies concepts to solve the question/problem | Applies concepts with minor mistakes | Limited or partially correct application | No or incorrect application |
| **Completeness of Response** | **15** | Covers all required parts of the question | Covers most parts with minor omissions | Covers only some parts | Incomplete or missing response |
| **Clarity & Presentation** | **15** | Clear, concise, and well-organized answer | Understandable with minor issues | Somewhat unclear or poorly structured | Difficult to understand |

# 1. Naïve Bayes Classifier: Feature Sets and Laplace Smoothing

## Aim

Implement a Naïve Bayes classifier on a real-world-style classification dataset, compare feature sets, and study the independence assumption and Laplace smoothing.

## Theory / Principle

Naïve Bayes uses Bayes' theorem: $P(C|x) \propto P(C)\prod_j P(x_j|C)$. It assumes that features are conditionally independent once the class is known. This assumption simplifies probability estimation, but correlated features may be counted as if they supplied separate evidence. For discrete/count data, additive (Laplace) smoothing estimates a conditional probability as (count + $\alpha$)/(class total + $\alpha$ × number of possible values). With $\alpha$=1, an unseen feature value receives a non-zero probability; without smoothing ($\alpha$=0), a zero count can make the likelihood for a class zero.

## Procedure

1) Load the breast-cancer dataset and split it into stratified training and test sets. 2) Fit Gaussian Naïve Bayes using all features and a smaller feature set. 3) Print accuracy, precision, recall and the confusion matrix for each feature set. 4) On discrete/count data, fit Multinomial Naïve Bayes with $\alpha$=0 (no smoothing) and $\alpha$=1 (Laplace smoothing), then compare the same metrics. 5) Discuss whether correlated features and unseen values may affect estimated class probabilities.

## Python implementation

```
from sklearn.datasets import load_breast_cancer, load_digits
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB, MultinomialNB
from sklearn.metrics import accuracy_score, precision_score, recall_score, confusion_matrix
import warnings

# A. Compare different feature sets using continuous features
data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)

feature_sets = {
    "all 30 features": list(X.columns),
    "first 5 features": list(X.columns[:5]),
    "first 10 features": list(X.columns[:10])
}
for label, cols in feature_sets.items():
    model = GaussianNB().fit(Xtr[cols], ytr)
    pred = model.predict(Xte[cols])
    print("\\nFeature set:", label)
    print("Accuracy:", round(accuracy_score(yte, pred), 4))
    print("Precision:", round(precision_score(yte, pred, zero_division=0), 4))
    print("Recall:", round(recall_score(yte, pred, zero_division=0), 4))
    print("Confusion matrix:\\n", confusion_matrix(yte, pred))

# B. Compare no smoothing (alpha=0) with Laplace smoothing (alpha=1)
# Digits pixels are non-negative count/intensity features.
digits = load_digits()
Xd_tr, Xd_te, yd_tr, yd_te = train_test_split(
    digits.data, digits.target, test_size=0.25,
    stratify=digits.target, random_state=42)

for alpha in [0.0, 1.0]:
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        model = MultinomialNB(alpha=alpha).fit(Xd_tr, yd_tr)
    pred = model.predict(Xd_te)
    print("\\nalpha =", alpha, "(0=no smoothing, 1=Laplace)")
    print("Accuracy:", round(accuracy_score(yd_te, pred), 4))
    print("Macro precision:", round(precision_score(
        yd_te, pred, average="macro", zero_division=0), 4))
    print("Macro recall:", round(recall_score(
        yd_te, pred, average="macro", zero_division=0), 4))
```

## Result analysis and discussion

Compare the printed metrics across the three feature sets rather than assuming that more features always improve performance. A reduced set can remove redundant/noisy variables and lower computation, but may discard useful evidence. When features are correlated, the conditional-independence assumption can make the posterior too confident because related evidence is effectively counted more than once. For smoothing, compare $\alpha$=0 and $\alpha$=1 on the same digit split. Without smoothing, a zero training count can create a zero likelihood for a class-feature combination; Laplace smoothing avoids this by assigning a small non-zero probability. The metrics may improve, worsen, or remain similar depending on the dataset, so report the actual printed values. Probability estimates matter because the classifier chooses the class with the largest estimated posterior; poor estimates can change the predicted class, especially when class probabilities are close.

## Conclusion

Feature selection, conditional-independence violations and smoothing all affect estimated class probabilities and therefore classification. Use the measured accuracy, precision and recall to decide which configuration performs best on held-out data.

# 2. Maximum Likelihood Estimation (MLE) versus Bayesian Estimation

## Aim

Estimate distribution parameters from data using MLE, compare with Bayesian estimates, and examine the effect of sample size and prior assumptions.

## Theory / Principle

Maximum Likelihood Estimation (MLE) chooses parameter values that maximize the likelihood of the observed data. For normally distributed observations, the MLE of the mean is the sample mean and the MLE of variance uses divisor n. Bayesian estimation combines the likelihood with a prior. With known observation variance $\sigma^2$ and prior $\mu \sim N(\mu\blacksquare, \tau^2)$, the posterior mean is a precision-weighted average of $\mu\blacksquare$ and the sample mean, and posterior variance is $1/(1/\tau^2+n/\sigma^2)$. A prior therefore has relatively greater influence with small n.

## Procedure

1) Generate a reproducible sample from a normal distribution. 2) For n=5, 20, 100 and 200, calculate sample mean and MLE variance. 3) Calculate Bayesian posterior mean and variance under a stated prior. 4) Repeat using a different prior mean to show prior sensitivity. 5) Compare small and large samples and explain the consequences for reliability.

## Python implementation

```
import numpy as np
rng = np.random.default_rng(7)
true_mean, true_variance = 10.0, 4.0
sample = rng.normal(true_mean, np.sqrt(true_variance), size=200)

# Prior for the unknown mean: Normal(mu0, tau2); observation variance is known.
prior_variance = 9.0
for n in [5, 20, 100, 200]:
    x = sample[:n]
    sample_mean = x.mean()                      # MLE of mean
    mle_variance = np.mean((x - sample_mean)**2) # MLE variance: divide by n
    posterior_variance = 1 / (1/prior_variance + n/true_variance)

    print(f"\\nn={n}; sample mean/MLE mean={sample_mean:.3f}")
    print(f"MLE variance={mle_variance:.3f}; true variance={true_variance:.1f}")
    for prior_mean in [7.0, 15.0]:
        posterior_mean = posterior_variance * (
            prior_mean/prior_variance + n*sample_mean/true_variance)
        print(f"prior mean={prior_mean:.1f}: posterior mean={posterior_mean:.3f}, "
              f"posterior variance={posterior_variance:.3f}")
```

## Result analysis and discussion

The output reports the sample mean (the MLE of $\mu$), the MLE variance, and Bayesian posterior mean/variance for each sample size. With n=5, random sampling variation is relatively large and the two different prior means can produce noticeably different posterior estimates. As n increases, the sample contributes more precision, the posterior variance decreases, and the posterior mean generally moves closer to the sample mean; the two prior choices have less influence. The MLE variance uses divisor n; the unbiased sample variance instead uses n–1. Reliability improves with more representative independent observations, but a large biased or unrepresentative sample is not automatically reliable. A reasonable prior can stabilize estimates for small samples, while a poorly chosen prior can pull the estimate away from the true value.

## Conclusion

MLE is determined by the observed sample, whereas Bayesian estimates combine sample evidence with prior assumptions. Report sample size, estimated mean and variance, prior choice, and posterior uncertainty to explain reliability.

# 3. Expectation-Maximization (EM) for a Gaussian Mixture Model

## Aim

Implement EM through a Gaussian mixture model and study initialization, number of clusters, convergence and local optima.

## Theory / Principle

A Gaussian Mixture Model (GMM) represents data as a weighted mixture of Gaussian components. EM alternates between an E-step, which estimates the responsibility of each component for each observation, and an M-step, which updates component weights, means and covariance matrices. The log-likelihood should not decrease in exact EM iterations, but the algorithm can converge to a local optimum; initialization matters.

## Procedure

Create a two-dimensional dataset, fit GMMs with different component counts and random initializations, and compare convergence iterations and final log-likelihood. Use BIC/AIC alongside likelihood because likelihood usually increases as components are added.

## Python implementation

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.mixture import GaussianMixture
from sklearn.preprocessing import StandardScaler

X, _ = make_blobs(n_samples=600, centers=3, cluster_std=[1.0, 1.7, 0.7],
                  random_state=12)
X = StandardScaler().fit_transform(X)

for k in [2, 3, 4]:
    for seed in [0, 10, 42]:
        gmm = GaussianMixture(n_components=k, n_init=1, max_iter=300,
                              tol=1e-4, random_state=seed)
        gmm.fit(X)
        print(f"k={k}, seed={seed}, converged={gmm.converged_}, "
              f"iterations={gmm.n_iter_}, log_likelihood={gmm.score(X)*len(X):.2f}, "
              f"BIC={gmm.bic(X):.2f}, AIC={gmm.aic(X):.2f}")

# More robust initialization: several restarts; the best likelihood is retained.
robust = GaussianMixture(n_components=3, n_init=10, random_state=42)
robust.fit(X)
print("10-restart model:", robust.converged_,
      robust.n_iter_, robust.score(X)*len(X))
```

## Result analysis and discussion

Record convergence status, iteration count, total log-likelihood, AIC and BIC. Different seeds may reach different likelihoods, indicating sensitivity to initialization. More components can improve likelihood while overfitting; BIC/AIC penalize model complexity. More restarts, standardized features, reasonable component counts and checking covariance regularization can improve stability. Convergence means the stopping tolerance was met, not that the global optimum was found.

## Conclusion

EM is useful for latent-variable estimation and probabilistic clustering, but results should be checked across initializations and model sizes rather than trusting one run.

# 4. Bayesian Belief Network and Probabilistic Inference

## Aim

Construct a Bayesian network, perform inference under different evidence conditions, and assess how dependencies and conditional probabilities affect posterior beliefs.

## Theory / Principle

A Bayesian Belief Network is a directed acyclic graph whose nodes are random variables and whose edges encode conditional dependencies. The joint distribution factorizes as $P(X1,\ldots,Xn)=\prod i\ P(Xi|Parents(Xi))$. Evidence changes posterior probabilities through Bayes' theorem. A network does not mean every connected variable causes another; edges represent modeled conditional dependence.

## Procedure

Use a simple illustrative network: Rain → WetGrass and Sprinkler → WetGrass. Define priors and conditional probabilities, calculate P(WetGrass), then calculate P(Rain|WetGrass). Change the evidence or CPT values and compare.

## Python implementation

```
# Exact enumeration for a small Bayesian network:
# P(Rain)=0.2; P(Sprinkler)=0.3; assumed independent priors.
# P(Wet|Rain,Sprinkler)=0.99
# P(Wet|Rain,not Sprinkler)=0.90
# P(Wet|not Rain,Sprinkler)=0.80
# P(Wet|not Rain,not Sprinkler)=0.01
p_rain, p_sprinkler = 0.2, 0.3
def p_wet(rain, sprinkler):
    table = {(True, True): 0.99, (True, False): 0.90,
             (False, True): 0.80, (False, False): 0.01}
    return table[(rain, sprinkler)]

p_wet_total = 0.0
p_rain_and_wet = 0.0
for r in [False, True]:
    for s in [False, True]:
        pr = (p_rain if r else 1-p_rain) * (
             p_sprinkler if s else 1-p_sprinkler)
        joint_wet = pr * p_wet(r, s)
        p_wet_total += joint_wet
        if r:
            p_rain_and_wet += joint_wet

print("P(WetGrass) =", round(p_wet_total, 4))
print("P(Rain | WetGrass) =", round(p_rain_and_wet/p_wet_total, 4))

# Sensitivity experiment: change the rain prior and repeat.
for prior in [0.1, 0.2, 0.5]:
    p_w, p_rw = 0.0, 0.0
    for r in [False, True]:
        for s in [False, True]:
            pr = (prior if r else 1-prior) * (
                 p_sprinkler if s else 1-p_sprinkler)
            jw = pr * p_wet(r, s)
            p_w += jw
            if r: p_rw += jw
    print("P(Rain)=", prior, "P(Rain|WetGrass)=", round(p_rw/p_w, 4))
```

## Result analysis and discussion

The output gives the marginal probability of wet grass and the posterior probability of rain when wet grass is observed. Compare posterior values across rain priors. Evidence is informative only to the extent that the likelihood of that evidence differs between hypotheses. In a larger network, exact inference can be expensive; approximate inference may be needed. Probabilities in a CPT must be non-negative and sum to one for each parent configuration.

## Conclusion

Bayesian networks provide a structured way to represent joint probability and update beliefs when evidence is observed. Results depend on the graph structure and the validity of the supplied conditional probabilities.

# 5. Concept Learning with Candidate Elimination

## Aim

Implement concept learning and study how hypothesis-space size, attributes and training examples affect the version space and generalization.

## Theory / Principle

Candidate Elimination maintains the most specific boundary S and most general boundary G of hypotheses consistent with the examples. Positive examples generalize S; negative examples specialize G. The version space contains hypotheses between the two boundaries. It can become empty if data are noisy or the target concept is not represented in the hypothesis space. Sample complexity generally grows with hypothesis-space complexity and the desired confidence/accuracy.

## Procedure

Use categorical attributes and labelled examples. Apply the S and G boundary updates for each example. Observe how the boundaries change and explain that adding attributes increases the possible hypothesis space. The implementation below uses a compact finite categorical hypothesis representation for teaching purposes.

## Python implementation

```
# Candidate Elimination for categorical attributes.
# '?' means any value; 'Ø' means the most-specific value.
def covers(h, x):
    return all(hi == '?' or hi == xi for hi, xi in zip(h, x))

def more_general_or_equal(h1, h2):
    return all(a == '?' or a == b for a, b in zip(h1, h2))

def candidate_elimination(X, y):
    n = len(X[0])
    S = [['Ø'] * n]
    G = [['?'] * n]
    domains = [sorted(set(row[j] for row in X)) for j in range(n)]

    for x, label in zip(X, y):
        if label == 1:  # positive example
            G = [g for g in G if covers(g, x)]
            new_S = []
            for s in S:
                if covers(s, x):
                    new_S.append(s)
                else:
                    candidate = [xi if si == 'Ø' else
                                 (si if si == xi else '?')
                                 for si, xi in zip(s, x)]
                    if all(v != 'Ø' for v in candidate):
                        new_S.append(candidate)
            S = new_S
            S = [s for s in S if any(more_general_or_equal(g, s) for g in G)]
        else:  # negative example
            new_G = []
            for g in G:
                if not covers(g, x):
                    new_G.append(g)
                else:
                    for j in range(n):
                        if g[j] == '?':
                            for val in domains[j]:
                                if val != x[j]:
                                    spec = g.copy()
                                    spec[j] = val
                                    if any(more_general_or_equal(spec, s) for s in S):
                                        new_G.append(spec)
            G = new_G
            G = [g for i, g in enumerate(G)
                 if not any(i != k and more_general_or_equal(G[k], g)
                            for k in range(len(G)))]
        print("example:", x, "label:", label, "S:", S, "G:", G)
    return S, G

X = [
    ['Sunny', 'Warm', 'Normal'],
    ['Sunny', 'Warm', 'High'],
    ['Rainy', 'Cold', 'High'],
    ['Sunny', 'Warm', 'Normal']
]
y = [1, 1, 0, 1]
S, G = candidate_elimination(X, y)
print("Final S:", S)
print("Final G:", G)
```

## Result analysis and discussion

The printed boundaries show how each labelled example constrains the hypotheses. More attributes create more possible combinations, which can increase sample requirements and computation. A finite hypothesis space can be enumerated; an infinite space cannot generally be exhaustively listed, so bounds and assumptions are needed. The mistake-bound model studies the maximum number of prediction errors before a learner converges under specified assumptions. Candidate Elimination is sensitive to contradictory/noisy labels; this simplified implementation is for categorical examples and should be checked carefully if extended.

## Conclusion

Concept learning is about narrowing a version space using labelled evidence. The number and diversity of examples, the representation of hypotheses and label quality determine whether a consistent concept can be identified.

# 6. Decision Tree: Entropy, Gini, Depth and Pruning

## Aim

Compare entropy and Gini splitting criteria and evaluate how tree depth and pruning influence classification performance and overfitting.

## Theory / Principle

A decision tree recursively splits the feature space. Entropy measures class uncertainty, H(S)=−Σp■log■p■; information gain is the reduction in entropy after a split. Gini impurity is 1−Σp■². Deep trees can fit training noise, while depth limits and pruning reduce complexity. Compare test metrics, not training accuracy alone.

## Procedure

Load a dataset, split into training and test sets, train trees using entropy and Gini, vary maximum depth, and compare accuracy and F1-score. Inspect training versus test scores to identify overfitting.

## Python implementation

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, f1_score
data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.25, stratify=data.target, random_state=42)

for criterion in ['gini', 'entropy']:
    for depth in [2, 4, None]:
        model = DecisionTreeClassifier(criterion=criterion, max_depth=depth,
                                       random_state=42)
        model.fit(X_train, y_train)
        train_acc = model.score(X_train, y_train)
        pred = model.predict(X_test)
        print(criterion, "depth=", depth,
              "train_acc=", round(train_acc, 3),
              "test_acc=", round(accuracy_score(y_test, pred), 3),
              "test_F1=", round(f1_score(y_test, pred), 3))

# Cost-complexity pruning: choose alpha using validation/cross-validation.
base = DecisionTreeClassifier(random_state=42)
path = base.cost_complexity_pruning_path(X_train, y_train)
for alpha in path.ccp_alphas[::max(1, len(path.ccp_alphas)//6)]:
    model = DecisionTreeClassifier(random_state=42, ccp_alpha=alpha)
    model.fit(X_train, y_train)
    print("ccp_alpha=", round(alpha, 5),
          "test_accuracy=", round(model.score(X_test, y_test), 3))
```

## Result analysis and discussion

Entropy and Gini often produce similar but not identical splits. An unrestricted tree may achieve near-perfect training accuracy while test performance is lower, indicating overfitting. Smaller depth and pruning may improve generalization but excessive pruning can underfit. The pruning parameter should ideally be selected using validation or cross-validation rather than repeatedly optimizing on the final test set.

## Conclusion

Decision trees are interpretable, but their complexity must be controlled. Compare criteria, depth and pruning using held-out metrics and the training–test performance gap.

# 7. K-Nearest Neighbors (KNN): K, Distance and Feature Scaling

## Aim

Evaluate the effect of K, Euclidean/Manhattan distance and feature scaling on KNN classification.

## Theory / Principle

KNN predicts a label from the majority class among the K closest training points. Euclidean distance is the square root of summed squared coordinate differences; Manhattan distance is the sum of absolute differences. Features with large numeric ranges can dominate distance, so scaling is usually important. Small K can be sensitive to noise; large K smooths the boundary but may underfit.

## Procedure

Use a classification dataset, compare K values and distance metrics, evaluate accuracy and confusion matrix, and compare unscaled versus standardized features. Keep scaling inside a pipeline to prevent data leakage.

## Python implementation

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, f1_score
data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.25, stratify=data.target, random_state=42)

for scale in [False, True]:
    for k in [1, 3, 5, 11, 21]:
        for metric in ['euclidean', 'manhattan']:
            knn = KNeighborsClassifier(n_neighbors=k, metric=metric)
            model = make_pipeline(StandardScaler(), knn) if scale else knn
            model.fit(X_train, y_train)
            pred = model.predict(X_test)
            print("scaled=", scale, "k=", k, "metric=", metric,
                  "accuracy=", round(accuracy_score(y_test, pred), 3),
                  "F1=", round(f1_score(y_test, pred), 3))
            if scale and k == 5 and metric == 'euclidean':
                print("Confusion matrix:\n", confusion_matrix(y_test, pred))
```

## Result analysis and discussion

Compare scaled and unscaled results. Standardization often improves KNN when features have different units/ranges, but it is not guaranteed to improve every dataset. K=1 may overfit local noise; large K may blur class boundaries. Euclidean and Manhattan distances can rank neighbours differently. Select K and metric through cross-validation, then report final test metrics once.

## Conclusion

KNN is simple and non-parametric but relies on meaningful distances. Scaling, K and distance metric should be treated as experimental parameters.

# 8. Support Vector Machine (SVM): Kernels, C and Gamma

## Aim

Compare linear, polynomial and RBF kernels and investigate the influence of C and gamma on decision boundaries and classification performance.

## Theory / Principle

An SVM finds a decision boundary with a large margin. The parameter C controls the penalty for training errors: large C prioritizes fitting training data; smaller C allows a wider margin. RBF gamma controls the influence radius of a training point; high gamma can create highly flexible boundaries. Kernel functions enable nonlinear separation without explicitly mapping data to high-dimensional features.

## Procedure

Standardize the features, fit several kernel/parameter combinations, and compare accuracy and precision. Use cross-validation for tuning in a full study.

## Python implementation

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, classification_report
data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.25, stratify=data.target, random_state=42)

configs = [
    ('linear', 1, 'scale'), ('poly', 1, 'scale'),
    ('rbf', 0.1, 0.01), ('rbf', 1, 'scale'),
    ('rbf', 10, 1.0)
]
for kernel, C, gamma in configs:
    model = make_pipeline(StandardScaler(),
                          SVC(kernel=kernel, C=C, gamma=gamma))
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print(kernel, "C=", C, "gamma=", gamma,
          "accuracy=", round(accuracy_score(y_test, pred), 3),
          "precision=", round(precision_score(y_test, pred), 3))
print("\nReport for final configuration:")
print(classification_report(y_test, pred, target_names=data.target_names))
```

## Result analysis and discussion

Scaling is essential for many SVM applications because margin and kernel calculations depend on feature magnitudes. A linear kernel is appropriate when the boundary is approximately linear; polynomial and RBF kernels can model nonlinear patterns. Excessive C or gamma can overfit, while values that are too small can underfit. Use stratified cross-validation and a parameter grid for robust selection.

## Conclusion

SVM performance is determined by the kernel, scaling and hyperparameters. Test-set performance and validation results should guide model selection.

# 9. K-Means Clustering: Cluster Count, Initialization and Scaling

## Aim

Cluster an unlabeled dataset and study how the number of clusters, initialization and feature scaling affect inertia and silhouette score.

## Theory / Principle

K-Means assigns each observation to its nearest centroid and iteratively updates centroids to minimize within-cluster sum of squared distances (inertia). It requires K in advance and is sensitive to scale, outliers and initialization. The silhouette score ranges from –1 to 1; higher values generally indicate better separation, though it is not suitable for every cluster shape.

## Procedure

Create an unlabeled dataset, scale features, fit K-Means for K=2…6 with different initializations, and record inertia and silhouette score. Use elbow and silhouette evidence together.

## Python implementation

```
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

X, _ = make_blobs(n_samples=500, centers=4, cluster_std=1.2, random_state=42)
X_scaled = StandardScaler().fit_transform(X)
for dataset_name, data in [('raw', X), ('scaled', X_scaled)]:
    for k in range(2, 7):
        for init in ['k-means++', 'random']:
            model = KMeans(n_clusters=k, init=init, n_init=10, random_state=42)
            labels = model.fit_predict(data)
            print(dataset_name, "k=", k, "init=", init,
                  "inertia=", round(model.inertia_, 2),
                  "silhouette=", round(silhouette_score(data, labels), 3))
```

## Result analysis and discussion

Inertia decreases as K increases, so the smallest inertia alone is not a valid criterion. Look for an elbow where marginal improvement slows and compare silhouette scores. K-means++ generally selects spread-out initial centers and can be more stable than random initialization, although multiple restarts are still recommended. Scaling matters when feature units/ranges differ. K-Means favours roughly spherical clusters and may perform poorly on irregular shapes.

## Conclusion

Cluster count should be selected using multiple diagnostics and domain knowledge. Scaling and initialization influence cluster assignments and repeatability.

# 10. Logistic Regression with L1 and L2 Regularization

## Aim

Compare unregularized/weakly regularized logistic regression with L1 and L2 penalties and examine accuracy, precision, recall, complexity and coefficient interpretability.

## Theory / Principle

Logistic regression estimates class probability using the logistic function σ(z)=1/(1+e■■). Regularization adds a penalty to the loss. L1 penalizes the sum of absolute coefficient values and can set some coefficients to zero; L2 penalizes squared coefficients and tends to shrink coefficients smoothly. In scikit-learn, smaller C means stronger regularization.

## Procedure

Standardize features, fit L1 and L2 logistic models with several C values, evaluate classification metrics, and count nonzero coefficients as a simple complexity indicator.

## Python implementation

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.25, stratify=data.target, random_state=42)

for penalty in ['l1', 'l2']:
    for C in [0.01, 0.1, 1, 10]:
        model = make_pipeline(StandardScaler(),
            LogisticRegression(penalty=penalty, C=C, solver='liblinear',
                               max_iter=5000, random_state=42))
        model.fit(X_train, y_train)
        pred = model.predict(X_test)
        lr = model.named_steps['logisticregression']
        nonzero = int(np.count_nonzero(lr.coef_))
        print(penalty, "C=", C,
              "accuracy=", round(accuracy_score(y_test, pred), 3),
              "precision=", round(precision_score(y_test, pred), 3),
              "recall=", round(recall_score(y_test, pred), 3),
              "F1=", round(f1_score(y_test, pred), 3),
              "nonzero_coefficients=", nonzero)
```

## Result analysis and discussion

With L1, stronger regularization often yields fewer nonzero coefficients, making the model more compact. L2 usually retains all features but reduces coefficient magnitudes. Too much regularization can underfit; too little can fit noise. Coefficient size is interpretable only in relation to feature scaling and coding. The default decision threshold of 0.5 may not be optimal when false positives and false negatives have different costs.

## Conclusion

Regularization controls model complexity and can improve generalization. Compare multiple metrics and coefficient patterns rather than relying on accuracy alone.

# 11. Random Forest: Number of Trees, Depth and Feature Randomness

## Aim

Evaluate how forest size and maximum tree depth affect performance and compare an ensemble with a single decision tree.

## Theory / Principle

Random Forest trains many decision trees on bootstrap samples and considers random subsets of features at splits. Averaging/voting reduces variance compared with a single unstable tree. More trees often stabilize predictions but increase computation. Deep trees may capture complex patterns; limiting depth can reduce overfitting.

## Procedure

Train forests with varying tree counts and depth, calculate accuracy, precision, recall and F1, and compare against a single decision tree using the same split.

## Python implementation

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.25, stratify=data.target, random_state=42)

models = [('DecisionTree', DecisionTreeClassifier(random_state=42))]
for trees in [10, 50, 100, 300]:
    for depth in [None, 5, 10]:
        models.append((f'RF trees={trees}, depth={depth}',
            RandomForestClassifier(n_estimators=trees, max_depth=depth,
                                   max_features='sqrt', random_state=42,
                                   n_jobs=-1)))
for name, model in models:
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print(name, "accuracy=", round(accuracy_score(y_test, pred), 3),
          "precision=", round(precision_score(y_test, pred), 3),
          "recall=", round(recall_score(y_test, pred), 3),
          "F1=", round(f1_score(y_test, pred), 3))
```

## Result analysis and discussion

Random Forest often generalizes better than a single tree because averaging reduces variance, but improvement is dataset-dependent. Increasing trees usually makes estimates more stable with diminishing returns. Restricting depth can reduce overfitting but may miss important interactions. Feature randomness decorrelates trees; max_features can be varied as another experiment. Use identical splits and metrics for a fair comparison.

## Conclusion

Random Forest balances predictive performance and robustness at the cost of reduced single-model interpretability and additional computation.

# 12. Principal Component Analysis (PCA) for Dimensionality Reduction

## Aim

Apply PCA to high-dimensional data and assess how the number of retained components affects explained variance, classification accuracy and computational efficiency.

## Theory / Principle

PCA transforms correlated features into orthogonal principal components ordered by decreasing variance. It is unsupervised and does not use labels when selecting components. Explained variance indicates retained variation, not necessarily retained class-discriminative information. Features should usually be standardized when units or scales differ.

## Procedure

Standardize the data, fit PCA with different component counts, transform the data, train the same classifier, and compare explained variance, test accuracy and training time. Use a pipeline to avoid leakage.

## Python implementation

```
import time
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.25, stratify=data.target, random_state=42)

for n in [2, 5, 10, 15, 20, 30]:
    pipe = Pipeline([('scale', StandardScaler()),
                     ('pca', PCA(n_components=n)),
                     ('clf', LogisticRegression(max_iter=5000, random_state=42))])
    start = time.perf_counter()
    pipe.fit(X_train, y_train)
    elapsed = time.perf_counter() - start
    pred = pipe.predict(X_test)
    variance = pipe.named_steps['pca'].explained_variance_ratio_.sum()
    print("components=", n, "variance_retained=", round(variance, 4),
          "accuracy=", round(accuracy_score(y_test, pred), 4),
          "fit_seconds=", round(elapsed, 4))
```

## Result analysis and discussion

As component count rises, explained variance cannot decrease, but classification accuracy may rise, fall or remain similar. Low-dimensional projections reduce storage and can speed later processing, while discarding components can lose useful information. Timing is affected by hardware and should be measured across repeated runs for a robust comparison. PCA components are linear combinations of original features, so interpretability is reduced.

## Conclusion

PCA is a useful compression technique, but component count should balance retained variance, predictive performance, computation and interpretability.

# 13. Gradient Descent: Learning Rate, Initialization and Convergence

## Aim

Implement gradient descent, vary learning rates and initial values, and observe convergence speed, iteration count and divergence.

## Theory / Principle

Gradient descent updates parameters in the direction opposite the gradient: $\theta(t+1)=\theta(t)-\eta\nabla J(\theta)$. The learning rate $\eta$ controls step size. Too small a rate converges slowly; too large a rate may overshoot or diverge. Initialization affects the starting point, especially for non-convex objectives. Momentum and adaptive optimizers can improve optimization in some settings.

## Procedure

Minimize a convex quadratic function, repeat for different learning rates and starting values, record the loss per iteration, and compare convergence. This controlled example isolates the learning-rate effect.

## Python implementation

```
import numpy as np
# J(w) = (w - 3)^2; gradient dJ/dw = 2(w - 3); optimum w=3, J=0.
def gradient_descent(w0, lr, max_iter=500, tol=1e-8):
    w = float(w0)
    history = []
    for i in range(max_iter):
        loss = (w - 3.0) ** 2
        history.append(loss)
        grad = 2.0 * (w - 3.0)
        w = w - lr * grad
        if abs(grad) < tol:
            return w, i + 1, history, True
        if not np.isfinite(w) or abs(w) > 1e12:
            return w, i + 1, history, False
    return w, max_iter, history, abs(2*(w-3)) < tol

for lr in [0.001, 0.05, 0.25, 0.6, 1.1]:
    for w0 in [-10, 0, 10]:
        w, iters, hist, converged = gradient_descent(w0, lr)
        print("lr=", lr, "initial=", w0, "final_w=", round(w, 6),
              "iterations=", iters, "converged=", converged,
              "final_loss=", round((w-3)**2, 8))
```

## Result analysis and discussion

For this quadratic, the gradient is zero at w=3. Small learning rates make gradual progress. A learning rate near the stable range converges faster; sufficiently large steps oscillate or diverge. The exact stable range depends on the curvature of the objective, so do not generalize a numeric learning rate to all problems. Plotting the recorded loss histories makes convergence patterns visible. Momentum uses a running update direction; adaptive methods scale steps based on gradient history.

## Conclusion

Gradient descent performance depends on objective curvature, learning rate and initialization. Monitor loss and gradient magnitude, and use validation or stopping criteria to avoid wasted iterations.

# 14. Neural Network Classifier: Architecture and Activation Functions

## Aim

Train neural networks with different hidden-layer sizes and activation functions, then compare accuracy, loss and overfitting.

## Theory / Principle

A feed-forward neural network composes weighted transformations and nonlinear activation functions. Hidden-layer count and neuron count determine model capacity. ReLU is commonly effective for hidden layers; logistic/sigmoid and tanh have different output ranges and gradient behaviour. More capacity may improve training fit but can increase overfitting. Learning curves compare training and validation performance over epochs.

## Procedure

Standardize data, compare multiple MLP architectures and activations, and evaluate held-out accuracy and loss. The code reports convergence and test metrics; for learning curves, use validation data or plot loss_curve_ for each fitted model.

## Python implementation

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, log_loss
data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.25, stratify=data.target, random_state=42)

configs = [
    ((16,), 'relu'), ((32,), 'relu'), ((32, 16), 'relu'),
    ((32,), 'tanh'), ((64, 32), 'logistic')
]
for layers, activation in configs:
    model = make_pipeline(StandardScaler(),
        MLPClassifier(hidden_layer_sizes=layers, activation=activation,
                      max_iter=1500, early_stopping=True, random_state=42,
                      learning_rate_init=0.001))
    model.fit(X_train, y_train)
    prob = model.predict_proba(X_test)
    pred = model.predict(X_test)
    mlp = model.named_steps['mlpclassifier']
    print("layers=", layers, "activation=", activation,
          "accuracy=", round(accuracy_score(y_test, pred), 4),
          "log_loss=", round(log_loss(y_test, prob), 4),
          "iterations=", mlp.n_iter_, "converged_by_limit=", mlp.n_iter_ < 1500)

# Example loss-curve data for the last fitted model:
print("Last model loss values (first 10):", model.named_steps['mlpclassifier'].loss_curve_[:10])
```

## Result analysis and discussion

Compare test accuracy and log loss alongside iteration count and loss curves. Larger networks can learn more complex boundaries but require more computation and may overfit. Early stopping monitors an internal validation split and can limit unnecessary training. Neural-network results can vary with random initialization; use repeated seeds or cross-validation for stronger evidence. A warning that the maximum iterations were reached means convergence was not necessarily achieved.

## Conclusion

Network architecture and activation affect learning capacity and optimization. Select the simplest architecture that achieves reliable validation performance rather than assuming deeper is always better.

# 15. Hierarchical Clustering: Linkage Methods and Dendrogram Analysis

## Aim

Compare single, complete and average linkage and evaluate how linkage criteria influence cluster formation using dendrograms and silhouette scores.

## Theory / Principle

Agglomerative hierarchical clustering begins with each observation as a separate cluster and repeatedly merges clusters. Single linkage uses the minimum pairwise distance, complete linkage uses the maximum, and average linkage uses the average pairwise distance. Single linkage can form chaining structures; complete linkage tends to create compact clusters; average linkage is a compromise. A dendrogram visualizes the merge sequence and distances.

## Procedure

Scale an unlabeled dataset, fit agglomerative clustering for each linkage method, calculate silhouette scores for a chosen cluster count, and plot dendrograms using SciPy. Use the same distance metric and data for a fair comparison.

## Python implementation

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import linkage, dendrogram

X, _ = make_blobs(n_samples=180, centers=3, cluster_std=1.3, random_state=21)
X = StandardScaler().fit_transform(X)

for method in ['single', 'complete', 'average']:
    model = AgglomerativeClustering(n_clusters=3, linkage=method,
                                    metric='euclidean')
    labels = model.fit_predict(X)
    print(method, "silhouette=", round(silhouette_score(X, labels), 4))

# Dendrograms for a manageable sample:
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, method in zip(axes, ['single', 'complete', 'average']):
    Z = linkage(X, method=method, metric='euclidean')
    dendrogram(Z, ax=ax, no_labels=True, color_threshold=None)
    ax.set_title(method.capitalize() + " linkage")
    ax.set_ylabel("Merge distance")
plt.tight_layout()
plt.show()
```

## Result analysis and discussion

Compare silhouette scores and dendrogram merge heights. Single linkage may join clusters through a chain of nearby points, while complete linkage resists long, spread-out clusters. Average linkage measures average cross-cluster distances. Dendrogram cuts at different heights produce different cluster counts, so state the chosen cut or number of clusters. Silhouette score is only one diagnostic and should be combined with visual/domain interpretation.

## Conclusion

Hierarchical clustering reveals nested group structure without requiring a full flat partition at the start. Linkage choice changes the hierarchy and should be selected based on cluster shape and interpretability.